

# CogStay Hotel Management Solution

Complete Architecture, Cloud Deployment, MongoDB Atlas Migration & JWT OTP Security Guide

## 1. Executive Summary & Refactoring Overview

CogStay has been completely transformed into a highly scalable, enterprise-grade 3-tier architecture powered by .NET 10.0 and MongoDB Atlas. All legacy SQL Server and EF Core dependencies have been entirely decoupled and eliminated in favor of high-performance MongoDB drivers, cryptographic OTP verification (Email + SMS), HttpOnly cookie-secured JWT token authentication with refresh token rotation, and robust concurrency controls to prevent hotel room double-bookings.

## 2. Clean 3-Tier Layered Architecture

The project is structured into clear, decoupled layer libraries following Domain-Driven Design (DDD) principles:

- **CogStay.Domain:** Zero-dependency POCO entities, core business domain enums (RoomStatus, ReservationStatus, PaymentStatus, OtpType, StaffRole), and MongoDB identity attributes.
- **CogStay.Application:** DTO contracts, persistence interface definitions, business domain services (Auth, Guest, Room, Reservation, CheckIn, Billing, Housekeeping, Feedback), and OTP cryptographic handlers.
- **CogStay.Infrastructure:** MongoDB Atlas context (MongoDbContext), automated collection indexing, Mongo repositories, JWT token generation, and SMTP/SMS notification dispatchers.
- **CogStayAPI & CogStay MVC:** ASP.NET Core Web API serving REST endpoints with JWT authorization, and server-side MVC client maintaining secure HttpOnly token cookies.

| Layer Component      | Target Project         | Key Responsibilities & Technologies                                           |
|----------------------|------------------------|-------------------------------------------------------------------------------|
| Presentation Client  | CogStay (MVC)          | HttpOnly Cookie state, Razor Views, Form validation, HttpClient forwarding    |
| REST API Service     | CogStayAPI             | JWT Bearer Validation, Controller endpoints, CORS policies, Idempotent Seeder |
| Application Layer    | CogStay.Application    | Business workflows, Double-booking checks, Dual OTP hashing & logout          |
| Infrastructure Layer | CogStay.Infrastructure | MongoDB Driver (v3.11), JWT generator, SmtplibEmailService, Index creation    |
| Domain Core          | CogStay.Domain         | Domain POCO entities, Enums, BSON ObjectId representation, Zero dependencies  |

### 3. MongoDB Atlas Database Schema & Indexing Strategy

All data is stored in MongoDB Atlas collections using optimized BSON documents. Automated compound and unique indexes are initialized during startup by MongoClientContext:

- **Guests Collection:** Unique index on Email and PhoneNumber.
- **Staff Collection:** Unique index on Email.
- **Rooms Collection:** Unique index on RoomNumber.
- **Otps Collection:** Compound index on (Target, OtpType) and **TTL Index** on ExpiresAt for automated record cleanup.
- **RefreshTokens Collection:** Compound index on Token and **TTL Index** on ExpiresAt.

### 4. Double-Booking Prevention & Concurrency Controls

To prevent double-booking room allocations under concurrent requests, ReservationService executes atomic date-range overlap verification:

Filter = roomId == targetRoom AND Status != Cancelled AND CheckInDate < targetCheckOut AND CheckOutDate > targetCheckIn

If any overlapping reservation exists, the API immediately throws an InvalidOperationException returning 400 Bad Request.

### 5. JWT Authentication & Dual OTP Verification Flow

- **Password Security:** Uses ASP.NET Core IPasswordHasher<TUser>. Plaintext passwords are never stored.
- **Dual OTP Verification:** Upon guest registration, two 6-digit random codes are generated, hashed via SHA-256 into CodeHash, and dispatched to Email and Mobile SMS. OTP records feature a 10-minute expiration, 5 max attempt lockout threshold, and 60-second resend cooldown. Guest accounts are set to IsActive = false until both Email and Phone OTPs are verified.
- **JWT & Refresh Token Rotation:** Access tokens expire in 60 minutes. Refresh tokens are single-use random 64-byte cryptographic strings. Upon refresh, the previous refresh token is immediately revoked and replaced.

### 6. Hosting Feasibility Analysis: Azure App Service vs Vercel

| Evaluation Criterion | Native Azure App Service / Container Apps                   | Vercel Serverless Hosting                          |
|----------------------|-------------------------------------------------------------|----------------------------------------------------|
| Runtime Support      | Native .NET 10.0 Long-Term Support (LTS) runtime            | Unsupported (Vercel is Node/JS serverless first)   |
| Background Services  | Full support for hosted IHostedService / background seeders | Incompatible due to stateless 10s execution limits |
| Cookie Handling      | Full HttpOnly, Secure cookie session propagation            | Complex edge lambda header rewriting required      |
| MongoDB Connectivity | Persistent socket pool to MongoDB Atlas                     | High connection overhead on serverless cold starts |
| Recommendation       | RECOMMENDED (Production Ready)                              | NOT RECOMMENDED FOR .NET BACKEND                   |

## 7. Production Security & Rollback Checklist

1. **Environment Secrets:** Store MONGODB\_CONNECTION\_STRING, JWT\_SIGNING\_KEY, and SMTP credentials in Azure Key Vault or App Service Application Settings.
2. **Database Backups:** Enable MongoDB Atlas continuous automated point-in-time recovery (PITR).
3. **HTTPS Strict Enforcement:** Set ASPNETCORE\_ENVIRONMENT=Production and enforce HTTPS redirection and HSTS headers.
4. **CORS Lockdown:** Restrict CORS\_ALLOWED\_ORIGINS strictly to official client domain origins.
5. **Serilog Centralized Logging:** Configure Serilog structured JSON logging to Azure Application Insights.